

EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from

© Pearson Education

Hengzhao Yang

Zhifeng Zhu

Yajun Ha

https://profile.iiita.ac.in/bibhas.ghoshal/COA_2020/Lab/Verilog.pdf



上海科技大学
ShanghaiTech University

Verilog

What is Verilog

Verilog is a hardware description language.

Younger than VHDL

Improvements compared to VHDL:

- Simpler syntax
- Greater ease of use
- Tools with better usability, such as IDEs and simulation



Example of a Verilog Program

```
module adder #(parameter W = 8)
(
    input clk,
    input [W-1:0] inA, inB,
    output [W:0] out,
    output isOdd
);

    reg [W-1:0] regA, regB;
    reg [W:0] regOut;
    reg regOdd;
    wire [W:0] wireOut;

    assign wireOut = regA + regB;

    assign out = regOut;
    assign isOdd = regOdd;

    always@(posedge clk)
    begin
        regA <= inA;
        regB <= inB;
        regOut <= wireOut;
        regOdd <= out[0];
    end
endmodule
```

Describe a module called **adder** which can be used to generate a family of circuits according to parameter **W** (default value is 8.) not a good name, since it is more than an Adder.

Input/
Output Ports

Create physical wires (Even though it says "**reg**"!)

Generate an addition circuit,
Inputs are **regA** and **regB** wires.
Output is connected to **wireOut**.

Attach wire **out** to wire **regOut**;
Attach wire **isOdd** to wire **regOdd**;

Create four positive edged-clocked registers, attacked to clock **clk**.

The outputs of the registers should be attached to the **regA,regB,regOut, regOdd** wires.

Inputs of registers should be attached to **inA, inB, wireOut, out[0]** wires.



Verilog Program

- Verilog is **case sensitive**
- Every sentence ends with ; symbol
- Verilog comment rules:
 1. One line comment: //
 2. Block comment: /* */



Concept of Verilog “Module”

In Verilog, the basic unit of hardware is called a module.

- Module declaration
- Ports
- Internal Variables
- Behavioral Description
- Instantiation of Other Modules

```
module module_name (list_of_ports);  
  
    input/output declarations;  
  
    local net declarations;  
  
    parallel statements;  
  
endmodule
```

Note: Starting from Verilog-2001, input/output declarations can be defined directly in the module port list, so there's no need to declare them again inside the module body

Concept of Verilog “Module”

List of port, types

- input (Type is wire only)
- output (Type can be wire or reg)
- inout (Type is wire only)

```
module alu (  
    input  [3:0] a,  
    input  [3:0] b,  
    input  [1:0] sel,  
    output reg [3:0] result  
);
```

```
always @(*) begin  
    case (sel)  
        2'b00: result = a + b;  
        2'b01: result = a - b;  
        2'b10: result = a & b;  
        2'b11: result = a | b;  
    endcase  
end  
  
endmodule
```



Old Style (Verilog-1995) of port declaration:

Two examples of old port declaration:

```
module simpleand (f, x, y);  
    input x, y;  
    output f;  
    assign f = x & y;  
endmodule
```

Simple AND gate

```
module two_level (a, b, c, d, f);  
    input a, b, c, d;  
    output f;  
    wire t1, t2;  
    assign t1 = a & b;  
    assign t2 = ~ (c | d);  
    assign f = t1 ^ t2;  
endmodule
```

Two-level circuit

Note: Starting from Verilog-2001, input/output declarations can be defined directly in the module port list, so there's no need to declare them again inside the module body

Module: Declaration of Variables

A variable belongs to one of two data types:

- Wire

- Must be continuously driven
- Typically used to represent connections in continuous assignments and module instantiations
- Declared using the wire keyword

- Register

- Retains the last value assigned to it
- Often used to represent storage elements
- Declared using keywords like reg or integer



Module: Declaration of Variables

- The 'reg' declaration explicitly specifies the size.

```
reg x, y; // single-bit register variables
```

```
reg [15:0] bus; // 16-bit bus, bus[15] MSB
```

- For 'integer', it takes the default size, usually 32-bits.

Both reg and integer supports arithmetic operations.



Assign Statement: how to define constant Values

- The common values used in modeling hardware are:
 - 0 :: Logic-0 or FALSE
 - 1 :: Logic-1 or TRUE
 - x :: Unknown (or don't care)
 - z :: High impedance
- Initialization:
 - All unconnected nets set to 'z'
 - All register variables set to 'x'

In hardware, initial values are real values (logic-0 or 1), but may vary depending on hardware status.



Example: Declaration of Variables

Example 1: See how each variable is defined.

```
module example (  
    input  wire clk,  
    input  wire reset,  
    input  wire [7:0] a,  
    input  wire [7:0] b,  
    output wire [7:0] sum,  
    output reg [7:0] result  
);  
  
    // Declaration of wire (combinational)  
    wire [7:0] temp_wire;  
  
    // Declaration of reg (sequential logic)  
    reg [7:0] temp_reg;  
    reg [3:0] counter;  
    reg ready;  
  
    // Declaration of integer (for general counting or simulation)  
    integer i;
```

Example 2:

During synthesis, 'register' data type maps either to a 'wire' or to a 'storage cell' depending on the context under which a value is assigned.

```
module comb_example(  
    input wire a, b,  
    output reg y  
);  
    always @(*) begin  
        y = a & b; // purely combinational assignment  
    end  
endmodule
```

No storage needed. Although y is declared as reg, this maps to combinational logic (wire) during synthesis.



parameter, localparam, `define

- parameter is customizable constant.
- Use when you want users of your module to be able to customize values

```
module counter #(parameter WIDTH = 8) (  
    input clk,  
    input rst,  
    output reg [WIDTH-1:0] count  
);  
    always @(posedge clk or posedge rst)  
        if (rst)  
            count <= 0;  
        else  
            count <= count + 1;  
endmodule
```

```
counter #(.WIDTH(16)) my_counter (  
    .clk(clk),  
    .rst(rst),  
    .count(count)  
);
```

You can override
default parameters
during instantiation



parameter, localparam, `define

- localparam is module's internal constant.
- Local to module. Cannot be overridden.

```
module timer #(parameter TIMEOUT = 100) (  
    input clk,  
    output reg done  
);  
    localparam MAX_COUNT = TIMEOUT - 1;  
  
    reg [7:0] count;  
    always @(posedge clk) begin  
        if (count == MAX_COUNT)  
            done <= 1;  
        else  
            count <= count + 1;  
        end  
    endmodule
```



parameter, localparam, `define

- `define is preprocessor macro (like #define in C/C++)
- It is “Global Setting”: shared across **multiple files or modules**

```
`define DATA_WIDTH 8

module register (
    input  [`DATA_WIDTH-1:0] d,
    output [`DATA_WIDTH-1:0] q
);
    assign q = d;
endmodule
```

Assign Statement

- The assign statement is used for continuous assignments, typically in combinational logic.
- The assign statement can only be used with wire types.

```
assign a = b >> 1;
```

```
assign a = b << 3;
```

```
assign f = {a, b};
```

```
assign f = {a, 3'b101, b};
```

```
assign f = {x[2], y[0], a};
```

```
assign f = { 4{a} }; // replicate four times
```

```
assign f = {2'b10, 3{2'b01}, x};
```



Assign Statement: How to Define Constant Values

- A value may be specified in either the 'sized' or the 'un-sized' form.

- Syntax for 'sized' form:

- `<size>'<base><number>`

- Examples:

- `8'b01110011 // 8-bit binary number`

- `12'hA2D // 1010 0010 1101 in binary`

- `12'hCx5 // 1100 xxxx 0101 in binary`

- `25 // signed number, 32 bits`

- `1'b0 // logic 0`

- `1'b1 // logic 1`



Verilog Operators

- Arithmetic operators

* , $/$, $+$, $-$, $\%$

- Logical operators

$!$ $\rightarrow$ logical negation

$\&\&$ $\rightarrow$ logical AND

$||$ $\rightarrow$ logical OR

- Relational operators

$>$, $<$, $>=$, $<=$, $==$, $!=$

- Bitwise operators

$\sim$, $\&$, $|$, $\wedge$, $\sim\wedge$

- Shift operators

$>>$, $<<$

- Concatenation $\{ \}$
- Replication $\{ n \{ \} \}$
- Conditional

$\langle \text{condition} \rangle ? \langle \text{expression1} \rangle : \langle \text{expression2} \rangle$

```
wire [3:0] a = 4'b1100;
```

```
wire [3:0] b = 4'b1010;
```

```
wire [7:0] result = {a, b}; // result = 8'b11001010
```



Verilog Operators

<code>+ - ! ~ unary</code>	highest precedence
<code>* / %</code>	
<code>+ - (binary)</code>	
<code><< >></code>	
<code>< <= > ></code>	
<code>== != === !==</code>	
<code>& ~&</code>	
<code>^ ^~ ~^</code>	
<code> ~ </code>	
<code>&&</code>	
<code> </code>	
<code>?: conditional</code>	lowest precedence

Use parentheses to
enforce your
priority

Some Points in Verilog Operators

- The presence of a 'z' or 'x' in a *reg* or *wire* being used in an arithmetic expression results in the whole expression being unknown ('x').
- The logical operators (!, &&, | |) all evaluate to a 1-bit result (0, 1 or x).
- The relational operators (>, <, <=, >=, ~=, ==) also evaluate to a 1-bit result (0 or 1).
- Boolean *false* is equivalent to 1'b0
Boolean *true* is equivalent to 1'b1.



Conditional Assignments

Grammar:

`condition ? value_if_true : value_if_false;`

```
assign out = sel ? a : b;
```

```
assign out = (sel == 2'b00) ? a :  
              (sel == 2'b01) ? b :  
              (sel == 2'b10) ? c : d;
```



Sequential (procedural) Block

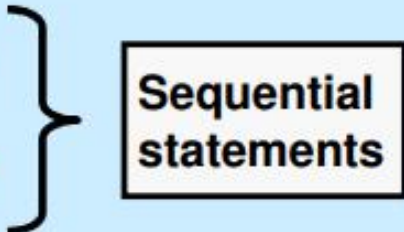
- The procedural block defines
 - A region of code containing *sequential* statements.
 - The statements execute in the order they are written.
- Two types of procedural blocks in Verilog
 - The “always” block
 - A continuous loop that never terminates.
 - The “initial” block
 - Executed once at the beginning of simulation (used in Test-benches).



Always: Sequential Statement

- A module can contain any number of “always” blocks, all of which execute concurrently.
- Basic syntax of “always” block:

```
always @ (event_expression)
begin
    statement;
    statement;
end
```



Sequential statements

- The @(event_expression) is required for both combinational and sequential logic descriptions.

CAD for VLSI



Always: Sensitivity list

- Clocked (Edge-Triggered) always block
- Combinational always block
- Level-Sensitive always block

```
always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 0;
    else
        q <= d;
end
```

```
always @(*) begin
    if (sel)
        out = a;
    else
        out = b;
end
```

```
always @(enable or d) begin
    if (enable)
        q = d;
end
```

Automatic Sensitivity List: When you use `@(*)`, Verilog automatically infers all the variables that are used inside the always block as part of the sensitivity list.



Always: Sequential Assignment

Sequential statements within procedural blocks (“always” and “initial”) can use two types of assignments:

- Blocking Assignment (using ‘=’)
- Non-Blocking Assignment (using ‘<=’)
(<= is intended to be used inside always.)

```
1  always @(posedge i_clock)
2  begin
3      r_Test_1 = 1'b1;
4      r_Test_2 = r_Test_1;
5      r_Test_3 = r_Test_2;
6  end
```

**Take 1 clock for
1'b1 to r_Test_3**

```
1  always @(posedge i_clock)
2  begin
3      r_Test_1 <= 1'b1;
4      r_Test_2 <= r_Test_1;
5      r_Test_3 <= r_Test_2;
6  end
```

**Take 3 clocks for
1'b1 to r_Test_3**

Do not mix = and <= in the same always block to avoid unpredictable behavior.

Inside Always: Sequential Statements

```
1. begin
    sequential_statements
end
2. if (expression)
    sequential_statement
    [else
        sequential_statement]
3. case (expression)
    expr:    sequential_statement
    .....
    default: sequential_statement
endcase
```

**begin...end
not required
if there
is only 1 stmt.**



Sequential Statements: Begin-end

- Used to group multiple statements.
- Similar to {} in C/C++/Java

```
always @(posedge clk) begin
    a <= b;
    c <= d;
end
```

```
always @(*) begin
    if (sel) begin
        y = a;
        z = b;
    end else begin
        y = c;
        z = d;
    end
end
```

```
always @(*) begin
    case (sel)
        2'b00: begin
            out = a;
            valid = 1;
        end
        2'b01: begin
            out = b;
            valid = 1;
        end
        default: begin
            out = 0;
            valid = 0;
        end
    endcase
end
```



Example: if else

if, **else if**, else

// A combinational logic example

```
module mux21 (in1, in0, s, f);  
  input in1, in0, s;  
  output f;  
  reg f;  
  
  always @ (in1 or in0 or s)  
    if (s)  
      f = in1;  
    else  
      f = in0;  
endmodule
```

```
always @(*) begin  
  if (sel) begin  
    y = a;  
    z = b;  
  end else begin  
    y = c;  
    z = d;  
  end  
end
```



Example: case

```
module ALU_4bit (f, a, b, op);  
    input [1:0] op;    input [3:0] a, b;  
    output [3:0] f;    reg [3:0] f;  
  
    parameter ADD=2'b00, SUB=2'b01,  
              MUL=2'b10, DIV=2'b11;  
  
    always @ (a or b or op)  
        case (op)  
            ADD : f = a + b;  
            SUB : f = a - b;  
            MUL : f = a * b;  
            DIV : f = a / b;  
        endcase  
endmodule
```

CAD for VLSI

```
always @(*) begin  
    case (sel)  
        2'b00: begin  
            out = a;  
            valid = 1;  
        end  
        2'b01: begin  
            out = b;  
            valid = 1;  
        end  
        default: begin  
            out = 0;  
            valid = 0;  
        end  
    endcase  
end
```



Instantiation of Other Modules

Module instantiation allows you to use pre-defined modules as components inside another module.

- Step 1: define module.
- Step 2: Instantiate module defined in step 1 in another module.

Port Mapping:

- positional association
 - named association
- .port_name(signal_name)**

```
module adder (  
    input wire a,    // 1-bit input 'a'  
    input wire b,    // 1-bit input 'b'  
    output wire sum  // 1-bit output 'sum'  
);  
  
    // Behavior of the adder (combinational logic)  
    assign sum = a ^ b; // XOR operation for addition of two 1-bit values  
  
endmodule
```

```
module top_module (  
    input wire a, b,    // Inputs for the top module  
    output wire sum     // Output sum from the top module  
);  
  
    // Instantiate the 'adder' module  
    adder adder_instance (  
        .a(a),          // Connect input 'a'  
        .b(b),          // Connect input 'b'  
        .sum(sum)        // Connect output 'sum'  
    );  
  
endmodule
```



Verilog Testbench

- **What is test bench?**
 - **A Verilog procedural block which executes only once.**
 - **Used for simulation.**
 - **Testbench generates clock, reset, and the required test vectors.**



Example Testbench

```
module shifter_toplevel;
    reg clk, clear, shift;
    wire [7:0] data;

    shift_register S1 (clk, clear, shift, data);
    initial
        begin
            clk = 0; clear = 0; shift = 0;
        end
    always
        #10 clk = !clk;
endmodule
```



Time Control in Testbench

``timescale <time_unit> / <time_precision>`

Example: ``timescale 1ns / 1ps`

- Time unit = 1 nanosecond (ns) → This is the base unit used for delays like #1.
- Time precision = 1 picosecond (ps) → This is how finely time can be resolved in simulation.

(time_value)

- Makes a block suspend for “time_value” time units.

```
`timescale 1ns / 1ps

module clk_gen_example;

    reg clk;

    initial begin
        clk = 0; // initialize clock
    end

    // Toggle every 5 ns → 10 ns period (100 MHz)
    always begin
        #5 clk = ~clk;
    end

endmodule
```

Simulator Directives

- **\$display**
 - Prints text or variables to stdout.
 - Syntax same as “printf”.
- **\$monitor**
 - Similar to \$display, but prints the value whenever the value of some variable in the given list changes.
- **\$finish**
 - Terminates the simulation process.
- **\$dumpfile**
 - Specify the file that will be used for storing the waveform.
- **\$dumpvars**
 - Starts dumping all the signals to the specified file.



Simulator Directives: \$display

\$display: print message to console at the time \$display is executed.

```
$display("At time %0t, signal1 = %b, signal2 = %d", $time, signal1, signal2);
```

%t: formats the current simulation time into the output

string. it is often used alongside the \$time system function.

%b: binary representation.

%d: decimal representation.

Simulator Directives: \$monitor

\$monitor: print message whenever any of them change

```
initial begin
    $monitor("Time=%0t | clk=%b | count=%d", $time, clk, count);
end
```

Output looks like:

```
Time=0 | clk=0 | count=0
Time=5 | clk=1 | count=0
Time=10 | clk=0 | count=1
Time=15 | clk=1 | count=1
Time=20 | clk=0 | count=2
```

```
initial begin
    $monitor("Time=%0t | count=%d", $time, count);
    #25 $monitoroff;
    #10 $display("Monitoring paused");
    #10 $monitoron;
    #20 $finish;
end
```

Simulator Directives: \$dumpfile, \$dumpvars

Record signals in a VCD (Value Change Dump) file.
You can visualize signals in a waveform viewer, such as GTKWave.

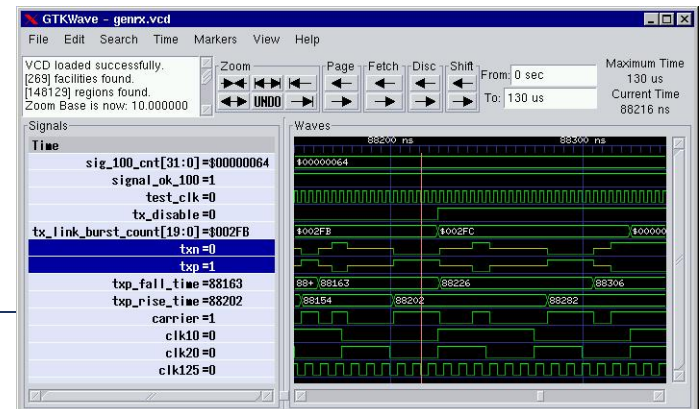
Step 1: call `$dumpfile("filename.vcd")`

- Specify the name of VCD file.

Step 2: call `$dumpvars(level, scope/signal)`

- Specify which signal to record.

GTKWAVE



Simulator Directives: \$dumpfile, \$dumpvars

Step 2: call \$dumpvars(**level**, scope/signal)

- Specify which signal to record.
- level=0: Dump all variables in the module and all submodules recursively.
- level=1,2,3..,n: Dump variables in the module and sub-submodules (n levels deep).

```
$dumpvars(0, testbench); // Dump top-level signals in 'testbench'
```

In case your file grows too large, record necessary signals only:

```
$dumpvars(0, testbench.signal1, testbench.signal2);
```



Assert & SystemVerilog

Standard Verilog does not natively support assert. You might want to learn SystemVerilog.

SystemVerilog is an extension of Verilog with many such verification features that allow engineers to verify the design using complex testbench structures and random stimuli in simulation.

In SystemVerilog:

```
assert (expression) else $fatal("Assertion failed!");
```

https://profile.iitita.ac.in/bibhas.ghoshal/COA_2020/Lab/Verilog.pdf

https://www.asic-world.com/verilog/verilog_one_day.html

https://hdlbits.01xz.net/wiki/Main_Page

HDLBits — Verilog Practice

HDLBits is a collection of small circuit design exercises for practicing digital hardware design using Verilog Hardware Description Language (HDL). Earlier problems follow a tutorial style, while later problems will increasingly challenge your circuit design skills.

Each problem requires you to design a small circuit in Verilog. HDLBits gives you immediate feedback on the circuit module you submit. Your circuit is checked for correctness by simulating with a set of test vectors and comparing it to our reference solution.

How to use HDLBits

1. Choose a problem: [Browse the problem set](#) or [go to the first problem](#) ○
2. Write a solution in Verilog
3. Submit, simulate, and debug if necessary

If you want to [track your progress](#) or move to another browser, [create a username and password](#) so you can [log in](#) from elsewhere.

Which exercises should I do?

The exercises are organized by topic and by approximately difficulty within each topic. Start first with the "Getting Started" section to get familiar with how to use HDLBits. Then start with the easier problems of each topic, and not in a strict top-to-bottom order. The "Verilog Language" section focuses more on using the Verilog syntax and language features, while the "Circuits" section focuses more on using Verilog to create circuits, so problems from these two categories should be done concurrently (practicing new language features while the circuits you create become more complex).